

Chapter 25 - Visualization with Matplotlib

We will study how to use `matplotlib` to generate data visualizations.

In []:

```
# %pip install matplotlib seaborn scikit-learn
```

Matplotlib shorthands

We will use the following aliases for `matplotlib` (`mpl`) and its MATLAB-style interface called `Pyplot` (`plt`).

In []:

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

Setting styles

In [59]:

```
plt.style.use('classic')
```

A first plot

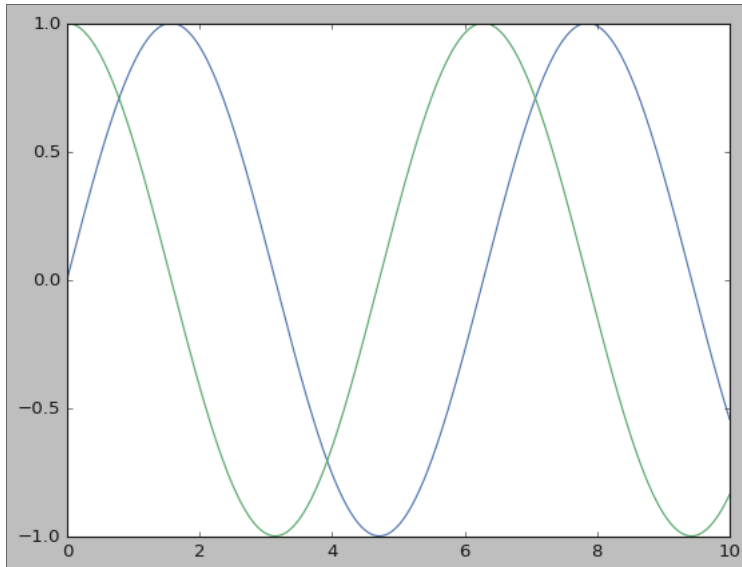
We will use `plt.show()` from within a script to display a plot. In IPython and Jupyter, you do not need to call `plt.show()`.

In [60]:

```
import numpy as np

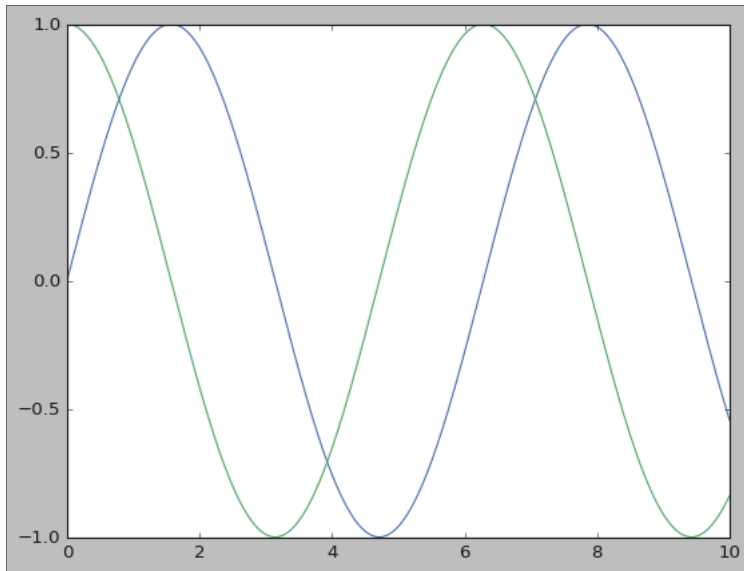
x = np.linspace(0, 10, 1000)

plt.plot(x, np.sin(x))
plt.plot(x, np.cos(x))
plt.show()
```



In [62]:

```
plt.plot(x, np.sin(x))
plt.plot(x, np.cos(x));
```



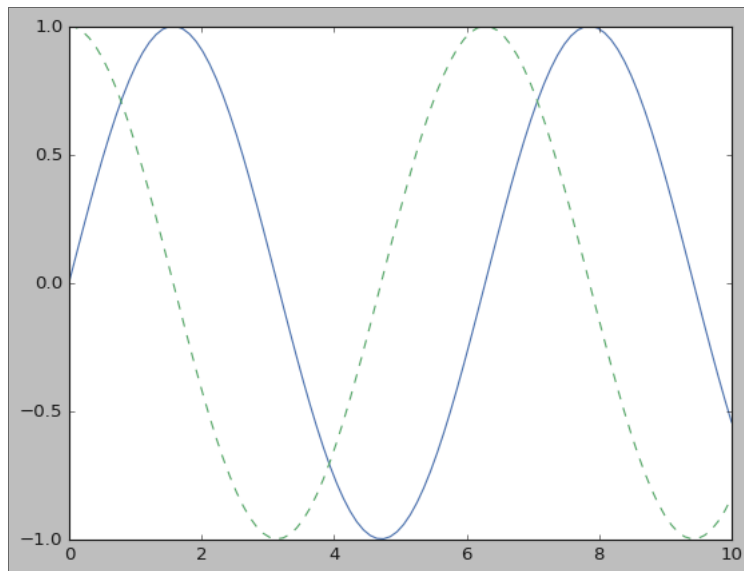
The `fig` object

You can store a plot into a figure object in matplotlib. This allows for greater control over the figure. For instance, we can save the figure to a file with:

In [63]:

```
x = np.linspace(0, 10, 100)

fig = plt.figure()
plt.plot(x, np.sin(x), '-')
plt.plot(x, np.cos(x), '--')
fig.savefig('my_figure.png')
```



The Pyplot interface

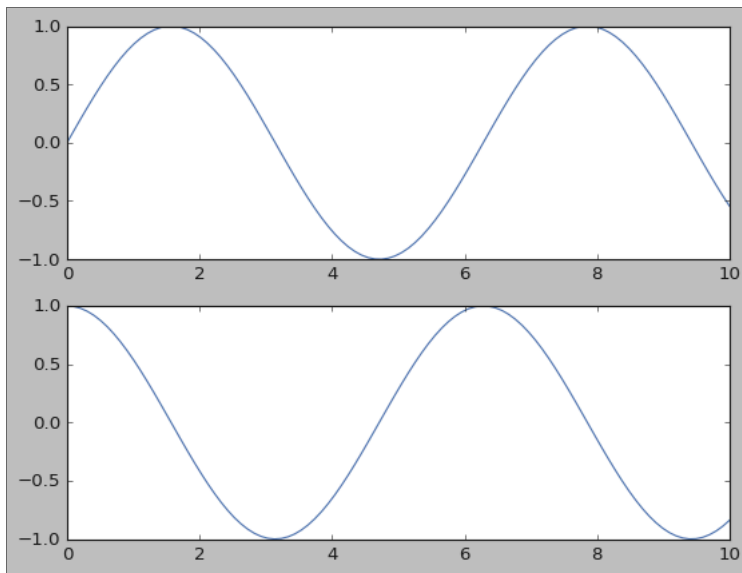
A quick and convenient interface to easily generate plots.

In [64]:

```
plt.figure() # create a plot figure

# create the first of two panels and set current axis
plt.subplot(2, 1, 1) # (rows, columns, panel number)
plt.plot(x, np.sin(x))

# create the second panel and set current axis
plt.subplot(2, 1, 2)
plt.plot(x, np.cos(x));
```



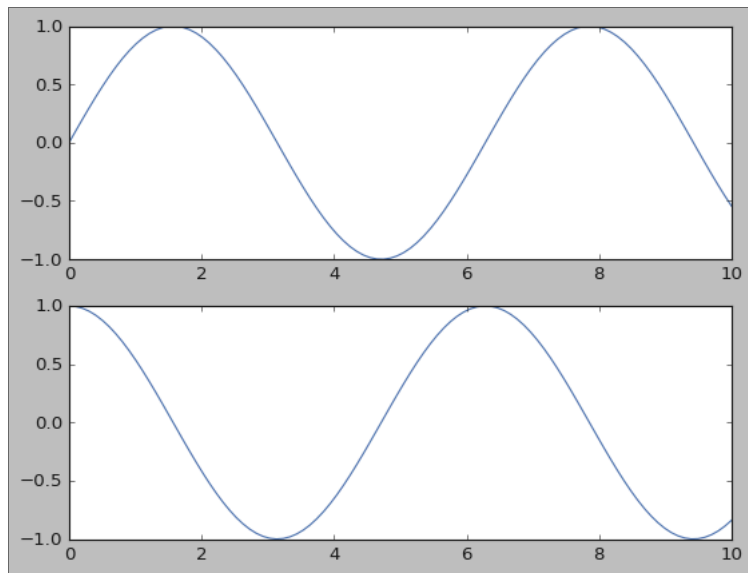
The object-oriented interface

An alternative to the Pyplot interface and is more versatile, albeit more complex. It is based on the **Figure** and **Axes** objects.

For simpler plots, you should go with Pyplot. For more elaborate figures, you might want to use the object-oriented interface. In many cases, the difference between using `plt` or `ax` is minimal.

In [65]:

```
# First create a grid of plots  
# ax will be an array of two Axes objects  
fig, ax = plt.subplots(2)  
  
# Call plot() method on the appropriate object  
ax[0].plot(x, np.sin(x))  
ax[1].plot(x, np.cos(x));
```

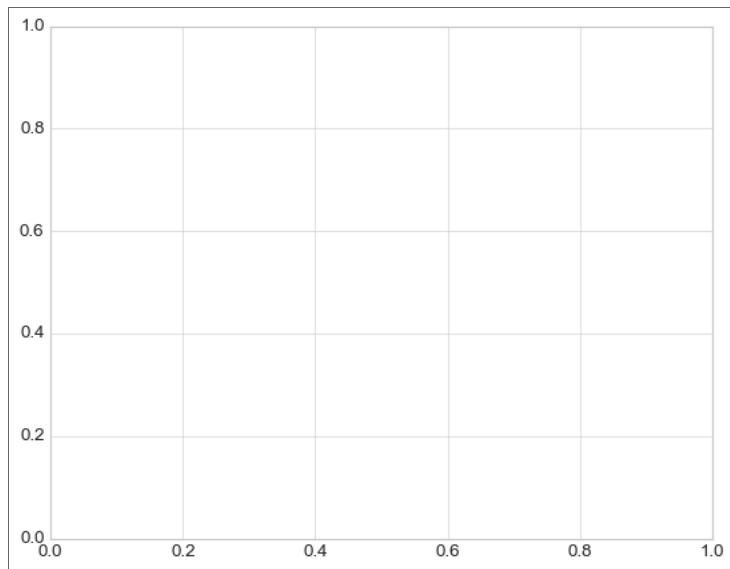


Chapter 26. Simple Line Plots

We will use the *seaborn whitegrid* style for the following plot. A **full list of styles** is available on matplotlib's website.

In [66]:

```
plt.style.use('seaborn-v0_8-whitegrid')  
  
# Create figure and axes  
fig = plt.figure()  
ax = plt.axes()
```



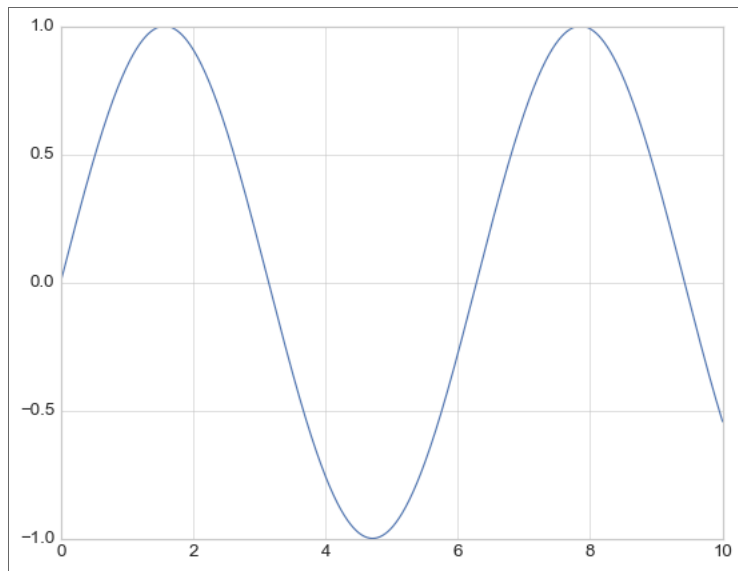
The `plot` method creates line plots. The following plots a simple sinusoid:

In [67]:

```
x = np.linspace(0, 10, 1000)
# Create figure and axes
fig = plt.figure()
ax = plt.axes()
ax.plot(x, np.sin(x))
```

Out[67]:

```
[<matplotlib.lines.Line2D at 0x129f14690>]
```



Plotting multiple lines

Every call to `plot` creates a new line. However, this does not immediately draw the figure. All plot calls are held back until the draw operation is triggered, either by IPython or by calling `show()`.

The `color` keyword in `plt.plot` allows us to choose the color of the line.

In [68]:

```
plt.plot(x, np.sin(x - 0), color='blue')      # specify color by name
plt.plot(x, np.sin(x - 1), color='g')        # short color code (rgbcmk)
plt.plot(x, np.sin(x - 2), color='0.75')     # grayscale between 0 and 1
plt.plot(x, np.sin(x - 3), color='#FFDD44')  # hex code (RRGGBB, 00 to FF)
plt.plot(x, np.sin(x - 4), color=(1.0,0.2,0.3)) # RGB tuple, values 0 to 1
plt.plot(x, np.sin(x - 5), color='chartreuse'); # HTML
```

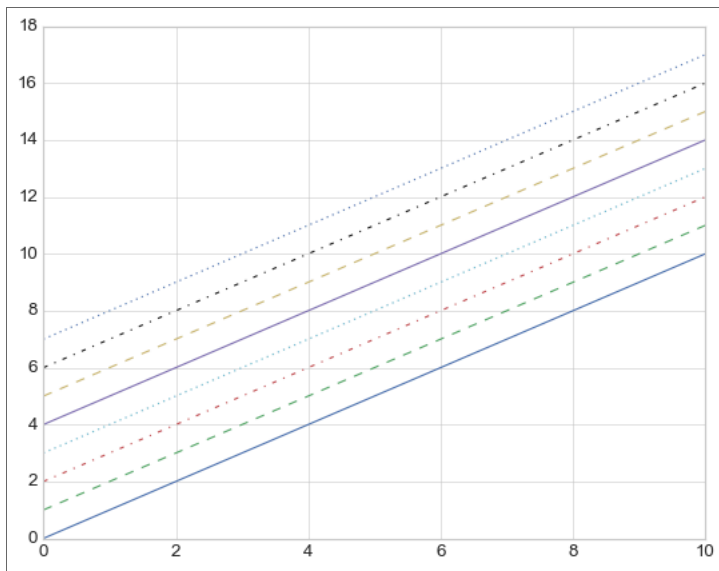


The line style can be defined using the `linestyle` keyword in `plt.plot`:

In [69]:

```
plt.plot(x, x + 0, linestyle='solid')
plt.plot(x, x + 1, linestyle='dashed')
plt.plot(x, x + 2, linestyle='dashdot')
plt.plot(x, x + 3, linestyle='dotted');

# For short, you can use the following codes:
plt.plot(x, x + 4, linestyle='-') # solid
plt.plot(x, x + 5, linestyle='--') # dashed
plt.plot(x, x + 6, linestyle='-.') # dashdot
plt.plot(x, x + 7, linestyle=':'); # dotted
```



You can use **style shorthands** by combining both line style and color into a string:

In []:

```
plt.plot(x, x + 0, '-g') # solid green
plt.plot(x, x + 1, '--c') # dashed cyan
plt.plot(x, x + 2, '-.k') # dashdot black
plt.plot(x, x + 3, ':r'); # dotted red
```

Axes limits

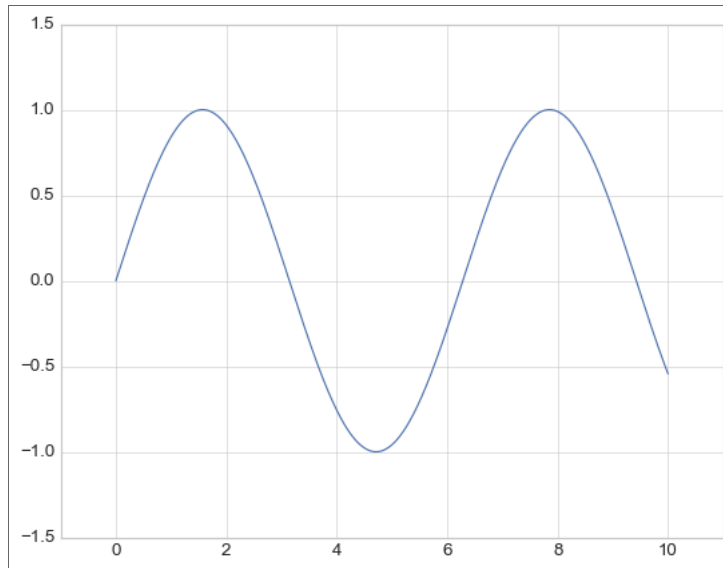
By default, matplotlib decides the axes limits based on the data. If you would like to manually adjust the limits, you can do so using `plt.xlim` and `plt.ylim`:

In [70]:

```
plt.plot(x, np.sin(x))
plt.xlim(-1, 11)
plt.ylim(-1.5, 1.5)
```

Out[70]:

(-1.5, 1.5)

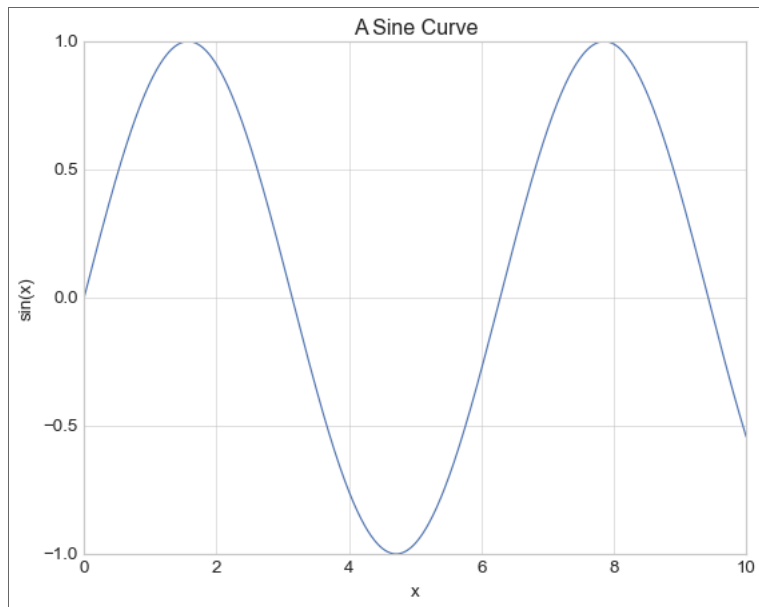


Plot labels

We can add titles, axis labels and legends:

In [71]:

```
plt.plot(x, np.sin(x))
plt.title("A Sine Curve")
plt.xlabel("x")
plt.ylabel("sin(x)");
```



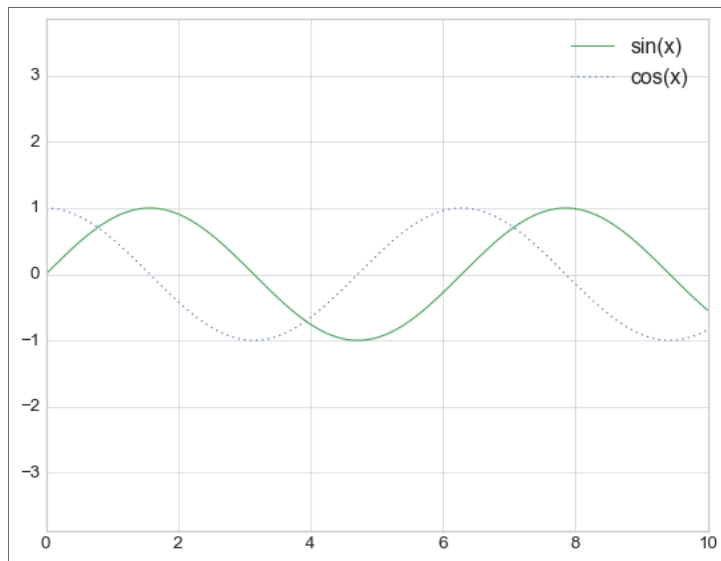
The `;` in the code above prevents the object to be output to IPython. It does not change the functioning of the code in any other way.

Adding a legend when multiple plots/lines are present. Note that `'-g'` and `':b'` are shorthand for line styles: solid green and dotted blue, respectively.

In [72]:

```
plt.plot(x, np.sin(x), '-g', label='sin(x)')
plt.plot(x, np.cos(x), ':b', label='cos(x)')
plt.axis('equal')

plt.legend();
```



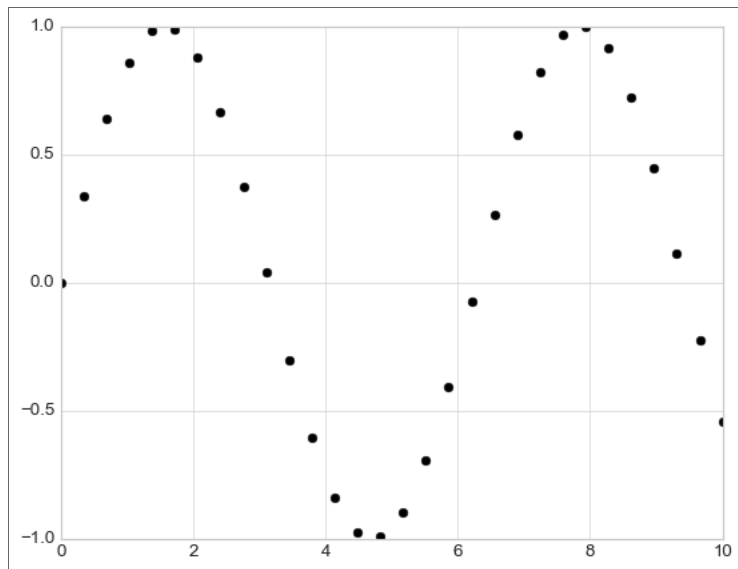
Chapter 27. Simple Scatter Plots

The `plot` function can also be used to generate scatter plots by using the style option `'o'`.

In [73]:

```
x = np.linspace(0, 10, 30)
y = np.sin(x)

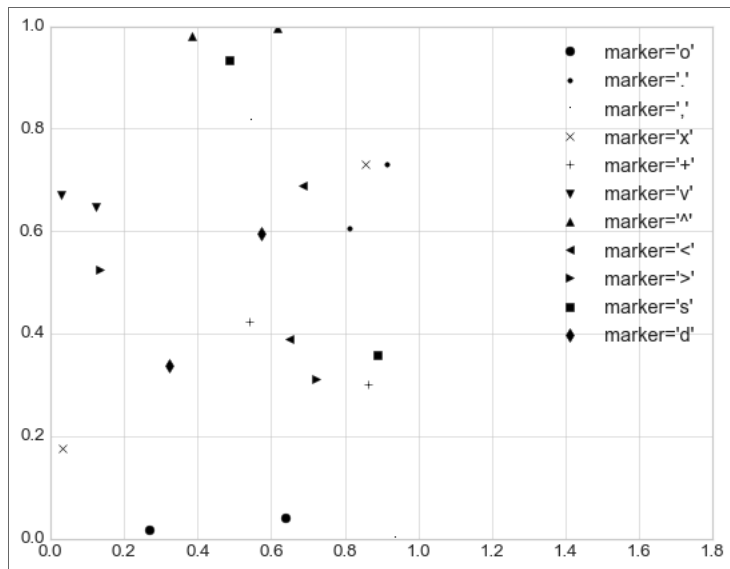
plt.plot(x, y, 'o', color='black');
```



You can choose different **markers** to appear in the plot:

In [74]:

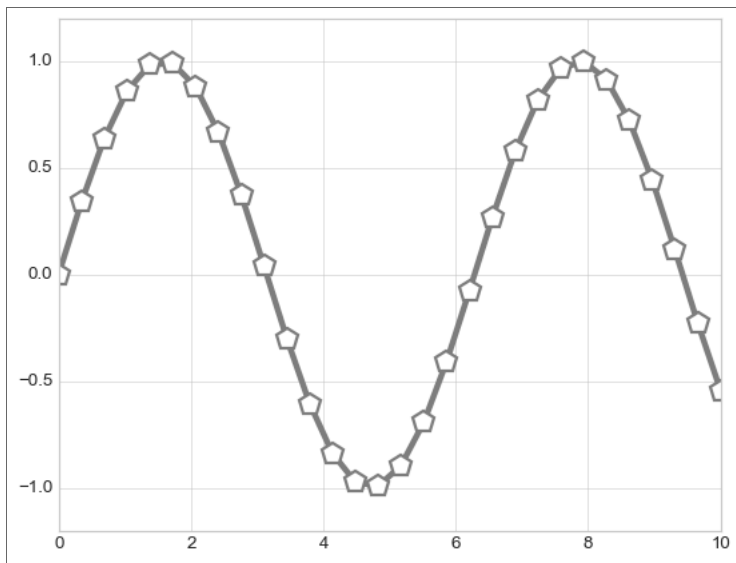
```
rng = np.random.default_rng(0)
for marker in ['o', '.', ',', 'x', '+', 'v', '^', '<', '>', 's', 'd']:
    plt.plot(rng.random(2), rng.random(2), marker, color='black',
             label="marker='{0}'".format(marker))
plt.legend(numpoints=1, fontsize=13)
plt.xlim(0, 1.8);
```



You can customize the markers and lines in a wide range of properties.

In [75]:

```
plt.plot(x, y, '-p', color='gray',
         markersize=15, linewidth=4,
         markerfacecolor='white',
         markeredgecolor='gray',
         markeredgewidth=2)
plt.ylim(-1.2, 1.2);
```



Scatter plots with `plt.scatter`

A more robust way to create scatter plots is using `plt.scatter`.

In []:

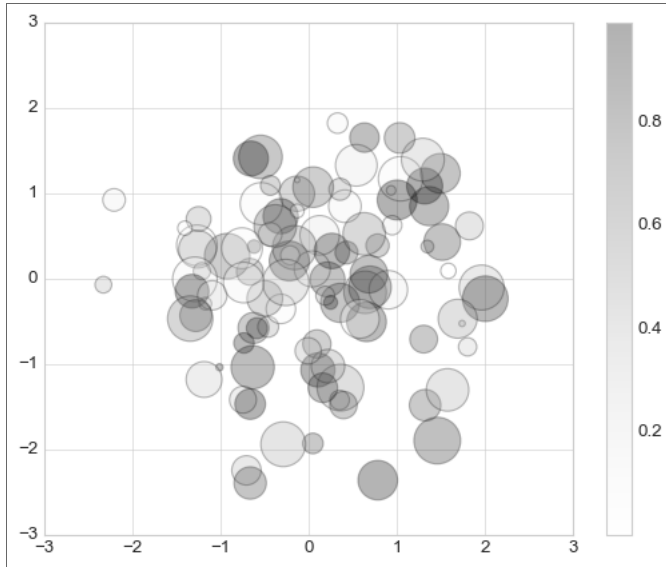
```
plt.scatter(x, y, marker='o');
```

`plt.scatter` allows you to control each marker's properties individually:

In [76]:

```
rng = np.random.default_rng(0)
x = rng.normal(size=100)
y = rng.normal(size=100)
colors = rng.random(100)
sizes = 1000 * rng.random(100)

plt.scatter(x, y, c=colors, s=sizes, alpha=0.3)
plt.colorbar(); # show color scale
```

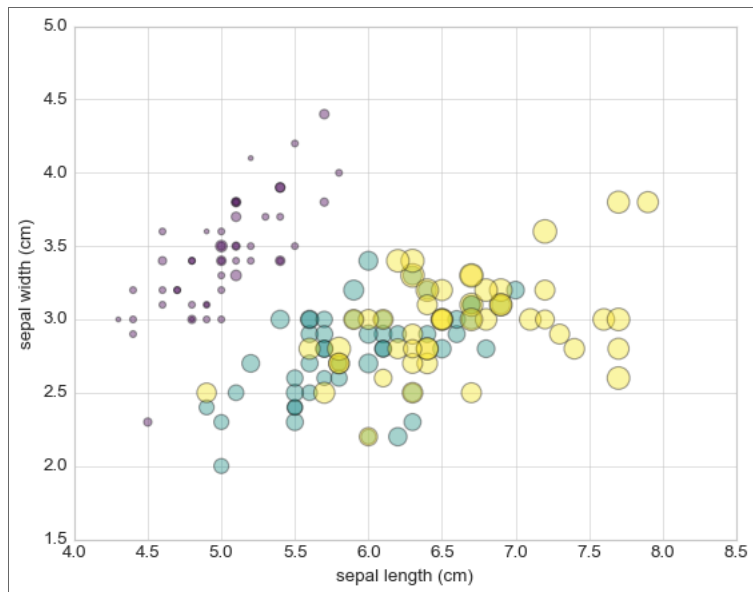


The following example uses the `iris` dataset from `sklearn`, where each sample comes from one of three different flower types with different petal size and sepal width.

In [77]:

```
from sklearn.datasets import load_iris
iris = load_iris()
features = iris.data.T

plt.scatter(features[0], features[1], alpha=0.4,
            s=100*features[3], c=iris.target, cmap='viridis')
plt.xlabel(iris.feature_names[0])
plt.ylabel(iris.feature_names[1]);
```



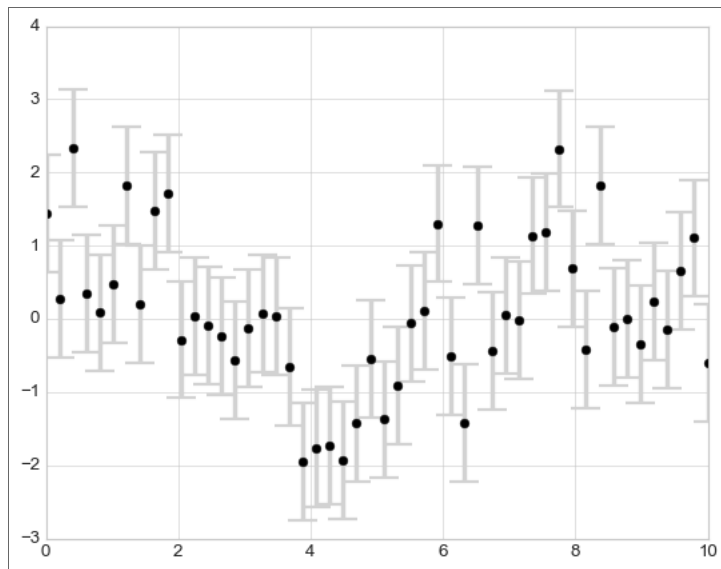
Plotting errorbars

If you have an error estimate and you would like to show it besides your plot, you can use `plt.errorbar`.

In [78]:

```
x = np.linspace(0, 10, 50)
dy = 0.8
y = np.sin(x) + dy * np.random.randn(50)

plt.errorbar(x, y, yerr=dy, fmt='o', color='black',
             ecolord='lightgray', elinewidth=3, capsize=10, capthick=2);
```

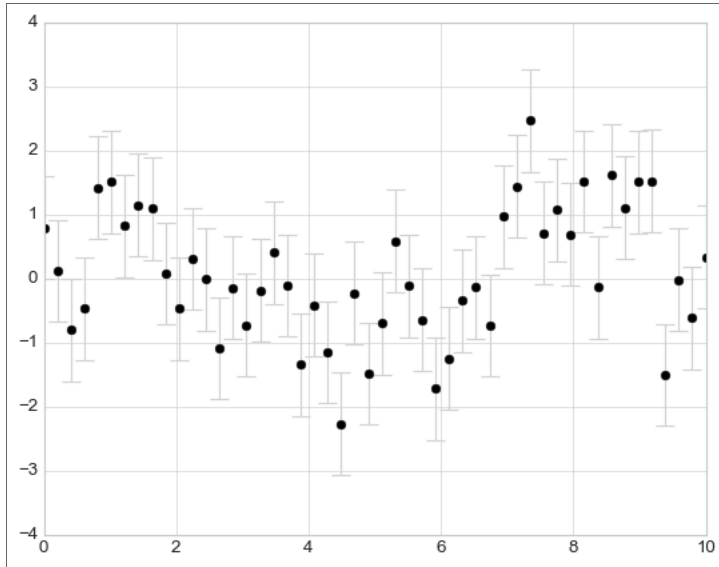


Try playing with the parameters and see what changes in the figure.

In [83]:

```
dy = 0.8
y = np.sin(x) + dy * np.random.randn(50)

plt.errorbar(x, y, yerr=dy, fmt='o', color='black',
             ecolord='lightgray', elinewidth=1, capsize=6, capthick=1);
```



Chapter 28. Density and Contour Plots

Contour plots can be used to display three-dimensional data in two dimensions. In Pyplot this is done using `plt.contour`.

In [84]:

```
# Let us define a function with two variables as follows:
def f(x, y):
    return np.sin(x) ** 10 + np.cos(10 + y * x) * np.cos(x)
```

The `plt.contour` function takes three arguments: a grid with `x`, `y` and `z` values. The `x` and `y` represent positions on the plane, the `z` values will be represented by the contour levels. You can think of it as the *height* of the plot at a point `x` and `y`.

To create this grid, we will use `np.meshgrid`.

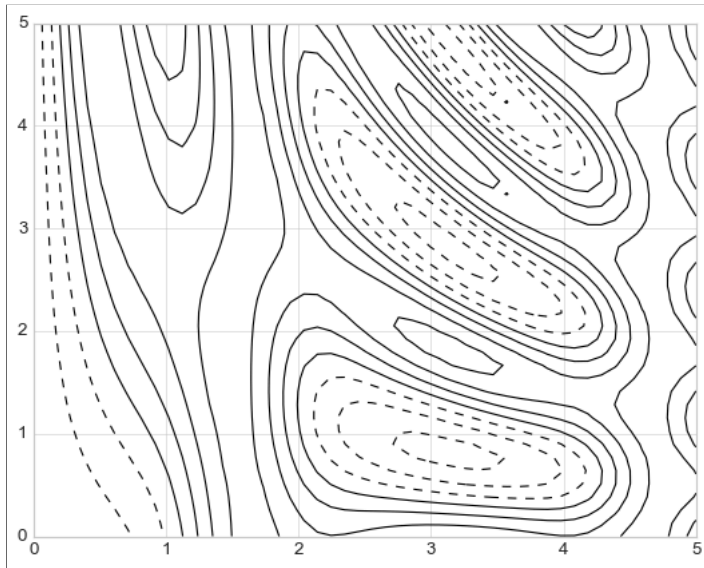
In [85]:

```
x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 40)
```

```
X, Y = np.meshgrid(x, y)
Z = f(X, Y)
```

In [86]:

```
plt.contour(X, Y, Z, colors='black');
```

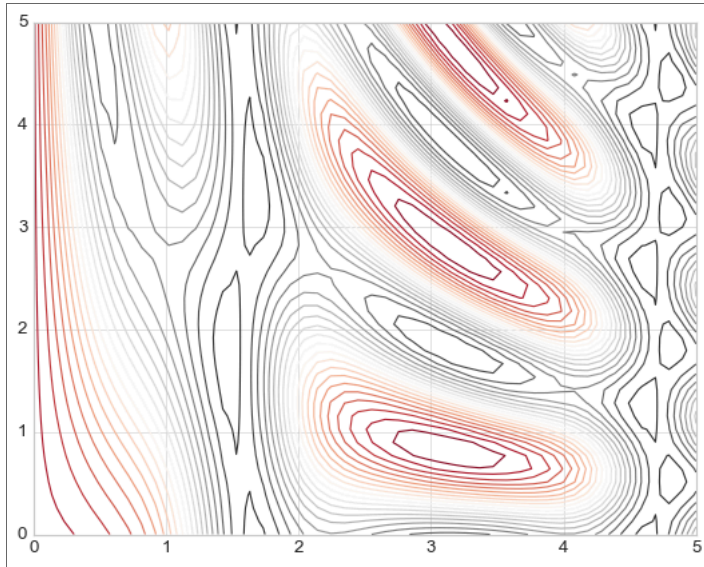


When using a single color, negative values are represented by dashed lines and positive values by solid lines.

We can use color-coded scales with the `cmap` argument. We can also specify the number of lines to be drawn.

In [87]:

```
plt.contour(X, Y, Z, 20, cmap='RdGy');
```

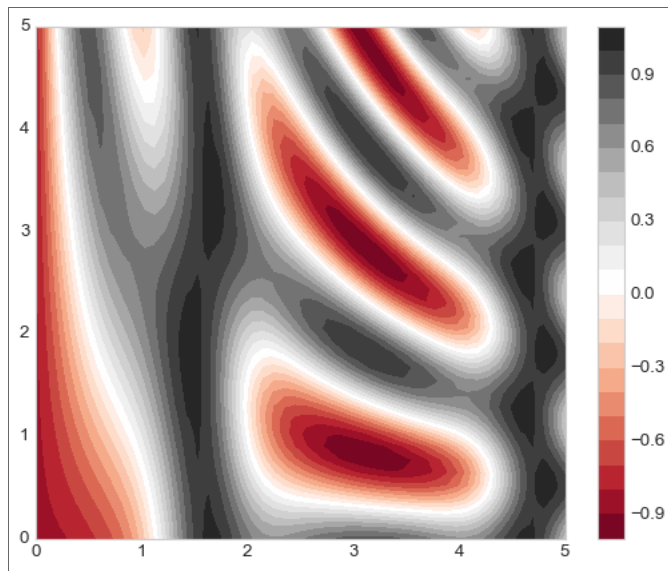


The contour is looking better, but the lines can be a bit distracting. We can use `plt.contourf` which *fills* the space between lines of a contour.

The result is similar to a heatmap, with colors indicating the values at each (x,y) coordinate.

In [88]:

```
plt.contourf(X, Y, Z, 20, cmap='RdGy')  
plt.colorbar();
```



Histograms, binnings and density

One of the most useful steps in understanding a dataset is looking at a histogram. The histogram will show the distribution of values for a certain variable in the data.

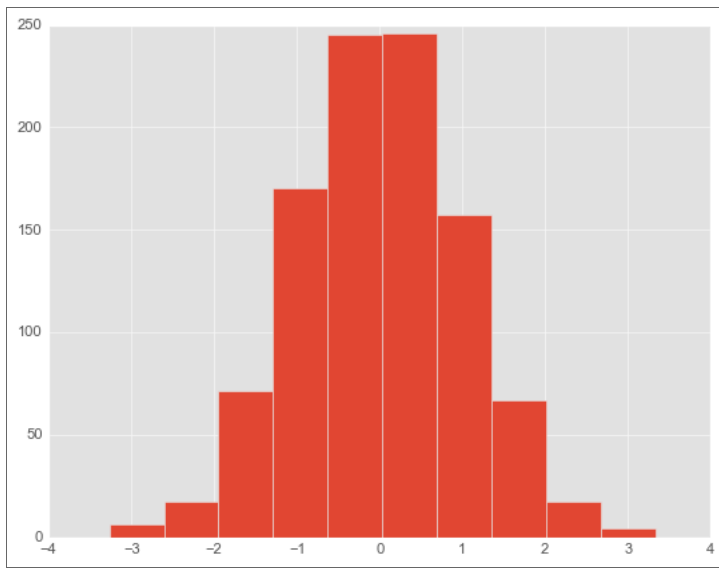
In [93]:

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('ggplot')

rng = np.random.default_rng(1701)
data = rng.normal(size=1000)
```

In [90]:

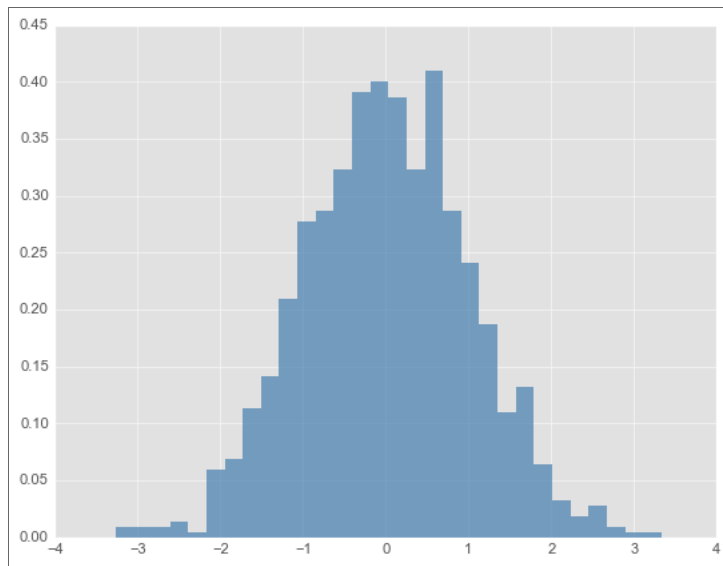
```
plt.hist(data);
```



The `hist` function allows customization of many parameters. The following is a histogram of the same data as above with tweaked parameters.

In [91]:

```
plt.hist(data, bins=30, density=True, alpha=0.7,  
         histtype='stepfilled', color='steelblue',  
         edgecolor='none');
```



This histogram has some transparency, controlled by `alpha`, it also uses the `stepfilled` style which creates filled rectangles for each bin.

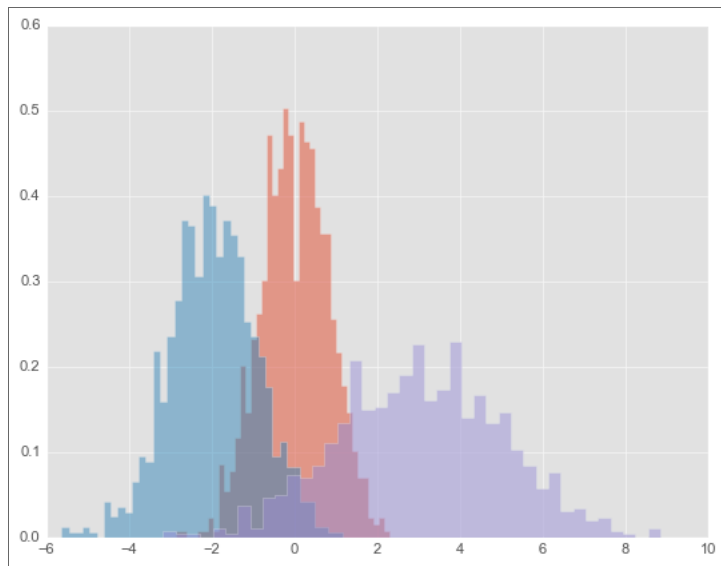
This setup is useful when comparing multiple distributions:

In [92]:

```
x1 = rng.normal(0, 0.8, 1000)
x2 = rng.normal(-2, 1, 1000)
x3 = rng.normal(3, 2, 1000)

kwargs = dict(histtype='stepfilled', alpha=0.5, density=True, bins=40)

plt.hist(x1, **kwargs)
plt.hist(x2, **kwargs)
plt.hist(x3, **kwargs);
```



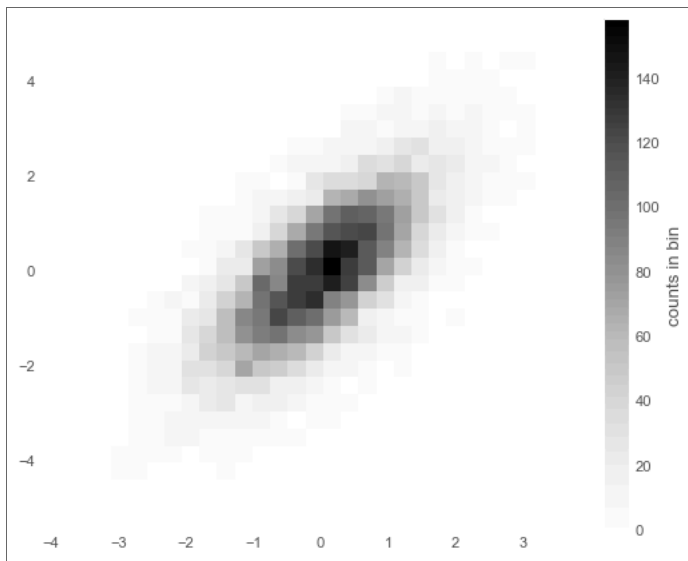
Two-dimensional histogram

In [94]:

```
# Generated 2D data
plt.style.use('ggplot')
mean = [0, 0]
cov = [[1, 1], [1, 2]]
x, y = rng.multivariate_normal(mean, cov, 10000).T
```

In [95]:

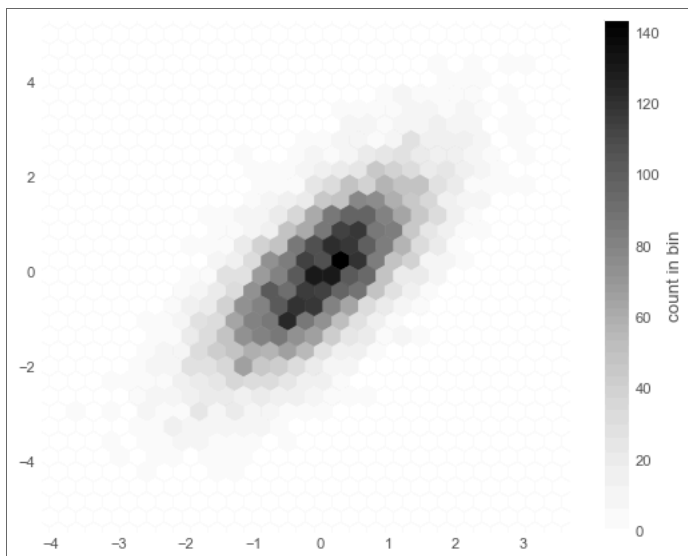
```
plt.hist2d(x, y, bins=30)
cb = plt.colorbar()
cb.set_label('counts in bin')
```



Using hexagonal bins instead of squares with `plt.hexbin`.

In [96]:

```
plt.hexbin(x, y, gridsize=30)  
cb = plt.colorbar(label='count in bin')
```



Chapter 36. Visualization with Seaborn

Matplotlib is a robust library for data visualization. Perhaps its major drawback is that it can be clunky when one wants to produce sophisticated plots. Moreover, it has no innate ability to integrate Pandas DataFrames, which are widely used in data applications.

This is where **Seaborn** comes in. Seaborn provides an API on top of Matplotlib enabling the use of high-level function for data visualization. It also fully integrates with Pandas structures.

By convention, we import Seaborn as `sns`.

In [97]:

```
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

sns.set_theme() # Set plot style
```

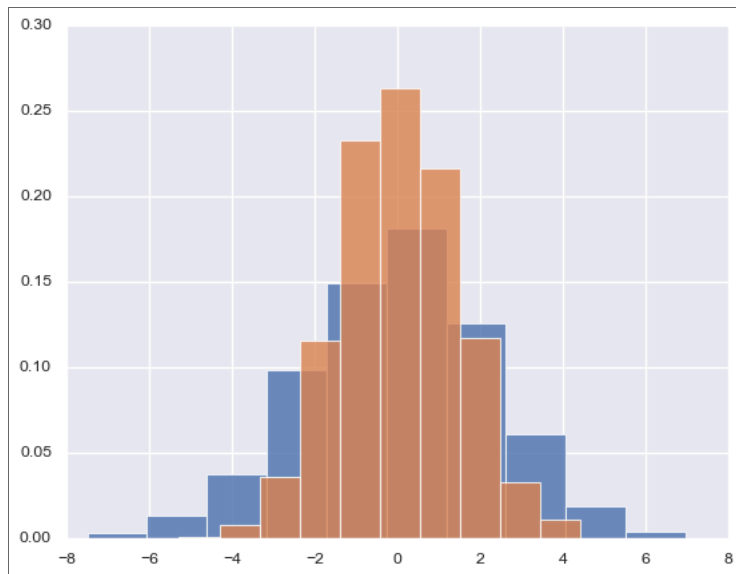
Seaborn plots

Histograms, KDE, and Densities

In [98]:

```
# Generate random variables, compare both distributions
data = np.random.multivariate_normal([0, 0], [[5, 2], [2, 2]], size=2000)
data = pd.DataFrame(data, columns=['x', 'y'])

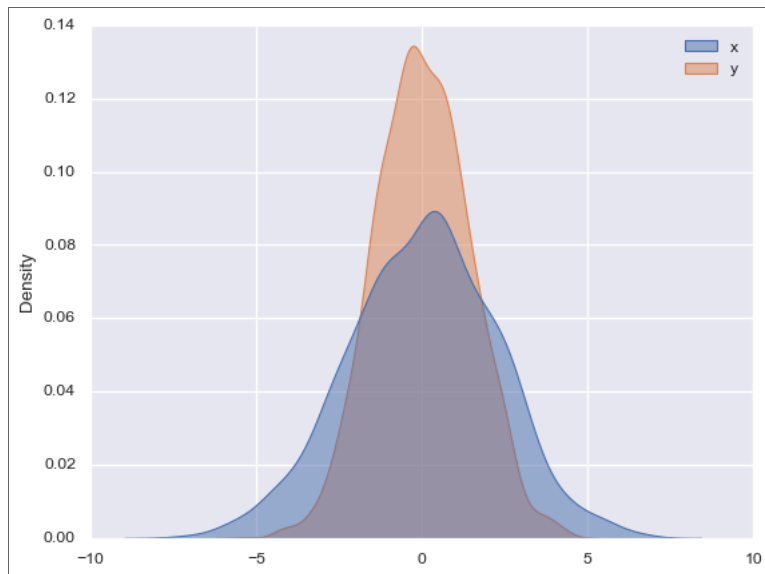
# Plot a histogram of both 'x' and 'y'
for col in 'xy':
    plt.hist(data[col], density=True, alpha=0.8)
```



A histogram will show bins as discrete (step) intervals. A **kernel density estimation** (KDE) is a smoothened estimate of the distribution. It is reasonable to use a KDE if your data is assumed to be continuous.

In [99]:

```
sns.kdeplot(data=data, fill=True, alpha=0.5);
```

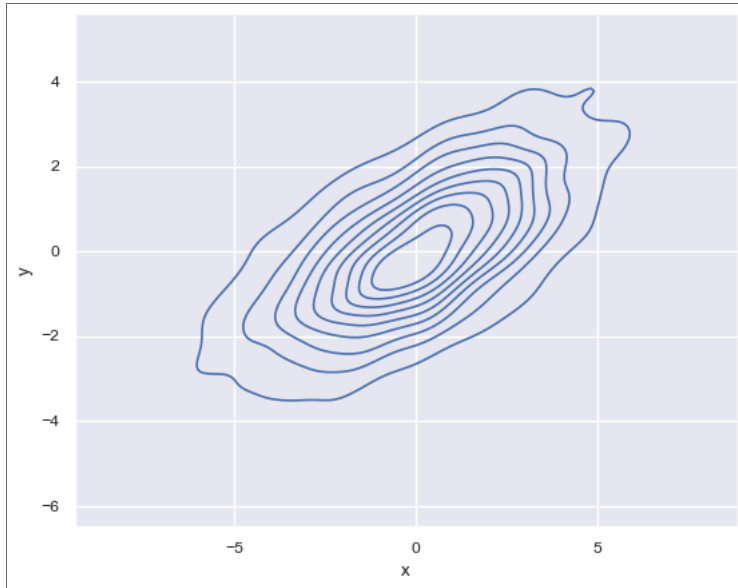


Note how Seaborn automatically figured out that your `data` contains two distributions in columns 'x' and 'y'.

If we pass arguments `x` and `y` to `kdeplot`, we can a two-dimensional KDE. Note that `x` and `y` are the *name* of the arguments, coincidentally equal to the data columns in this example.

In [100]:

```
sns.kdeplot(data=data, x='x', y='y');
```



Pair plots

Pair plots are a convenient way to explore potential correlations in your data when multiple variables are available.

We will use a `iris` dataset provided by Seaborn.

In [101]:

```
iris = sns.load_dataset('iris')  
iris.head()
```

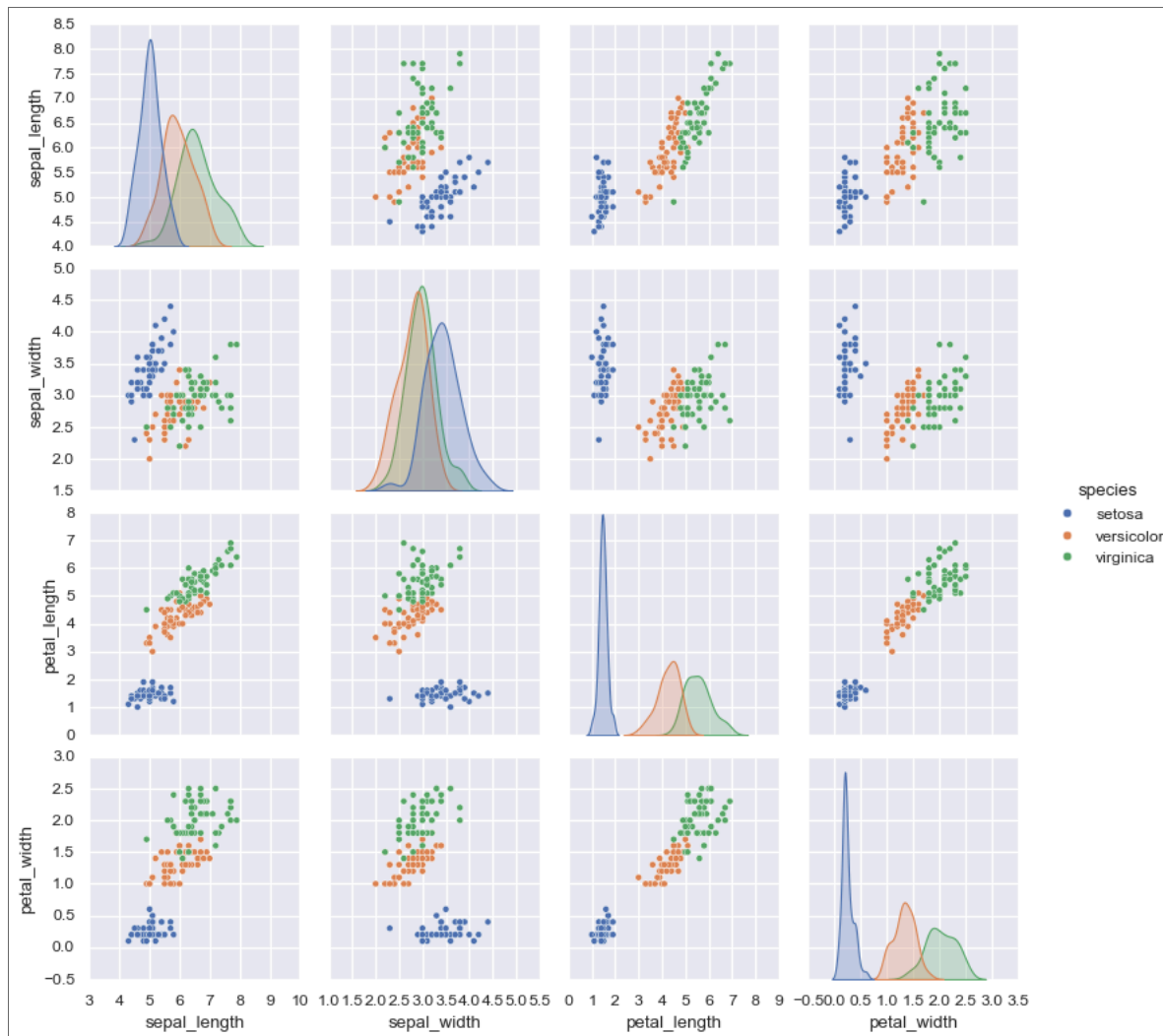
Out[101]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

The following will show a visualization of the relationships between each variable.

In [102]:

```
sns.pairplot(iris, hue='species', height=2.5);
```



Faceted Histograms

Faceted histograms break your data up into subsets based on categorical columns. Let us look at the `tips` dataset from Seaborn which describes tips received by male and female restaurant staff at lunch and at dinner.

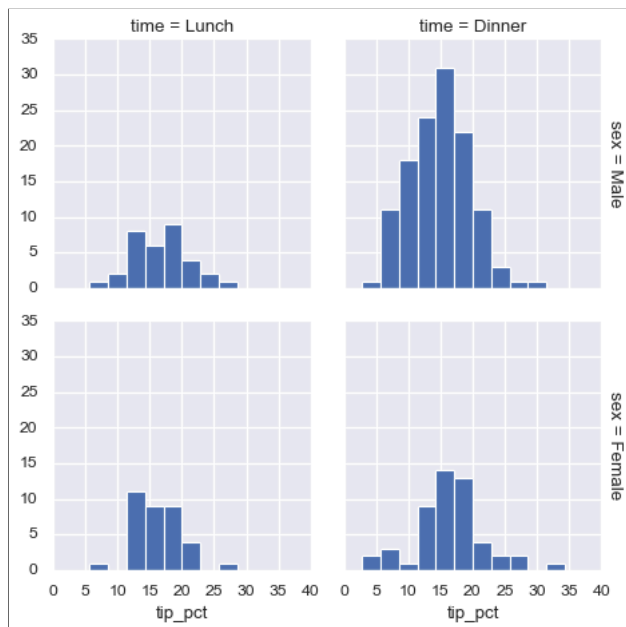
In [103]:

```
# Load dataset
tips = sns.load_dataset('tips')
tips.head()

# Calculate tip percentage
tips['tip_pct'] = 100*tips['tip']/tips['total_bill']
```

In [104]:

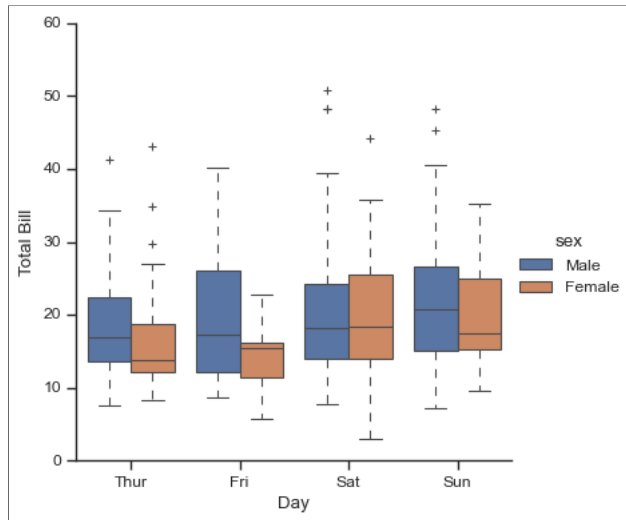
```
grid = sns.FacetGrid(tips, row='sex', col='time', margin_titles=True) # Create a FacetGrid where rows correspond to 's
grid.map(plt.hist, 'tip_pct', bins=np.linspace(0, 40, 15));
```



Categorical plots

In [105]:

```
with sns.axes_style(style='ticks'):
    g = sns.catplot(x="day", y="total_bill", hue="sex",
                    data=tips, kind="box")
    g.set_axis_labels("Day", "Total Bill");
```

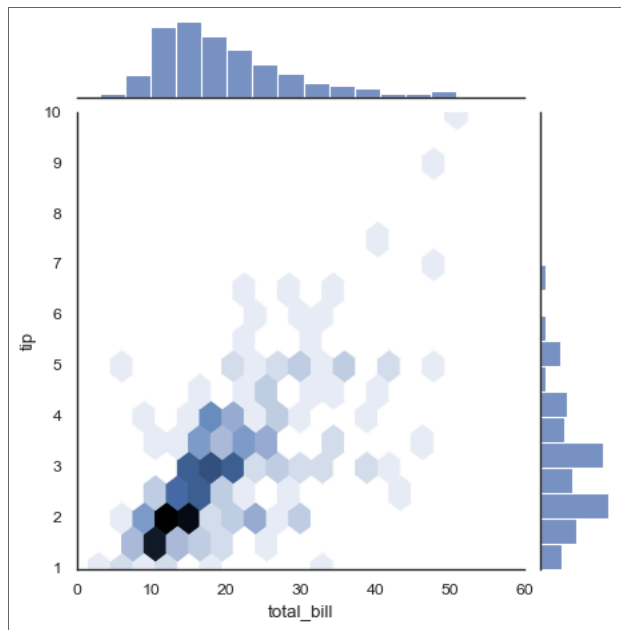


Joint distributions

Joint plots display the relation between two distributions in the data.

In [106]:

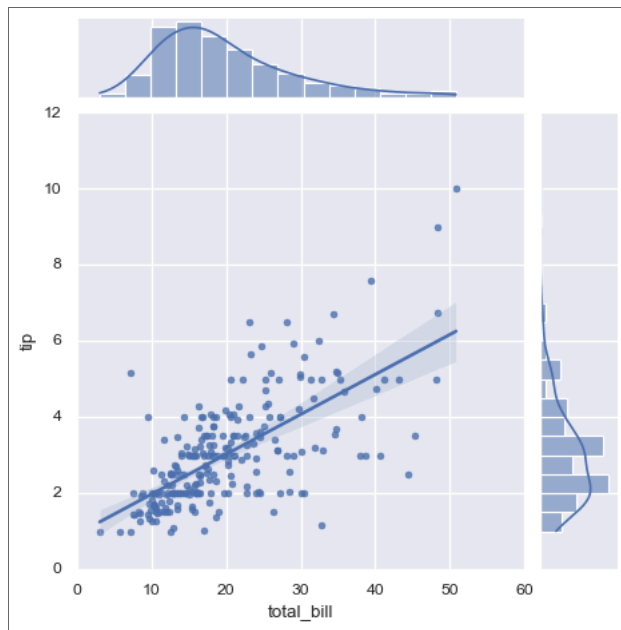
```
with sns.axes_style('white'):
    sns.jointplot(x="total_bill", y="tip", data=tips, kind='hex')
```



Joint plot can also do regression.

In [107]:

```
sns.jointplot(x="total_bill", y="tip", data=tips, kind='reg');
```



Bar plots

In [108]:

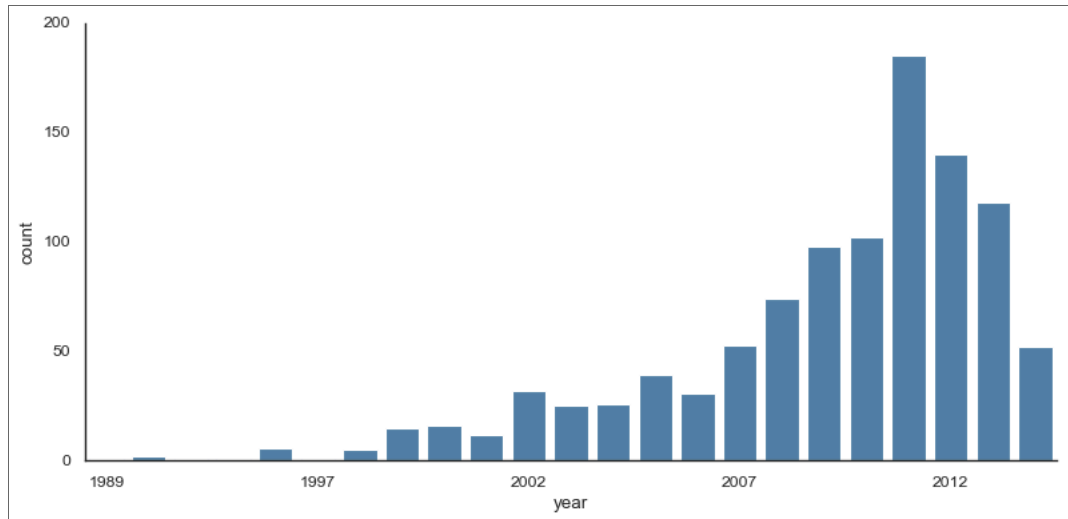
```
# Loading the planets dataset
planets = sns.load_dataset('planets')
planets.head()
```

Out[108]:

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007
4	Radial Velocity	1	516.220	10.50	119.47	2009

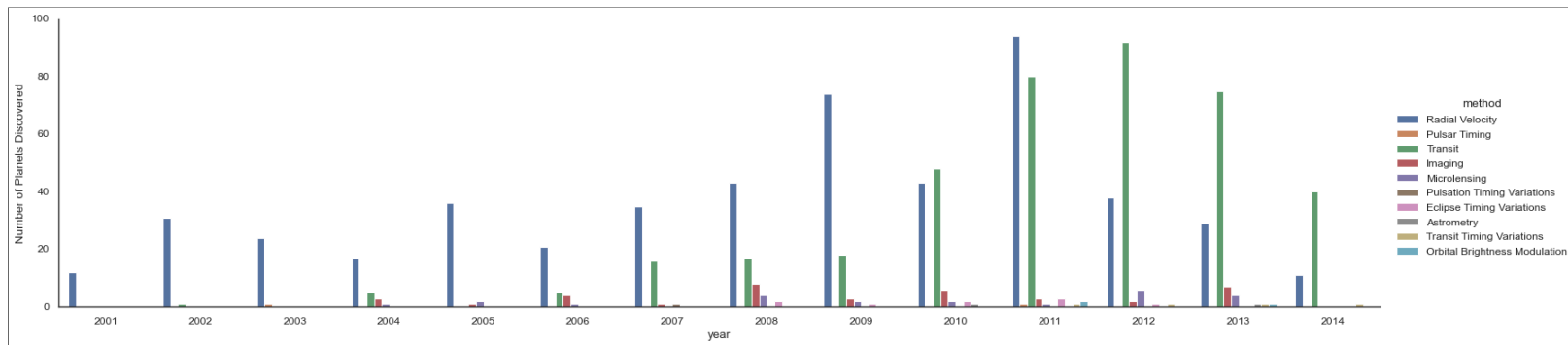
In [109]:

```
with sns.axes_style('white'):
    g = sns.catplot(x="year", data=planets, aspect=2,
                    kind="count", color='steelblue')
    g.set_xticklabels(step=5)
```



In [110]:

```
with sns.axes_style('white'):
    g = sns.catplot(x="year", data=planets, aspect=4.0, kind='count',
                    hue='method', order=range(2001, 2015))
    g.set_ylabels('Number of Planets Discovered')
```



Exploring the Taxis dataset

In [111]:

```
taxis = sns.load_dataset('taxis')
taxis['pickup'] = pd.to_datetime(taxis['pickup'])
taxis.head()
```

Out[111]:

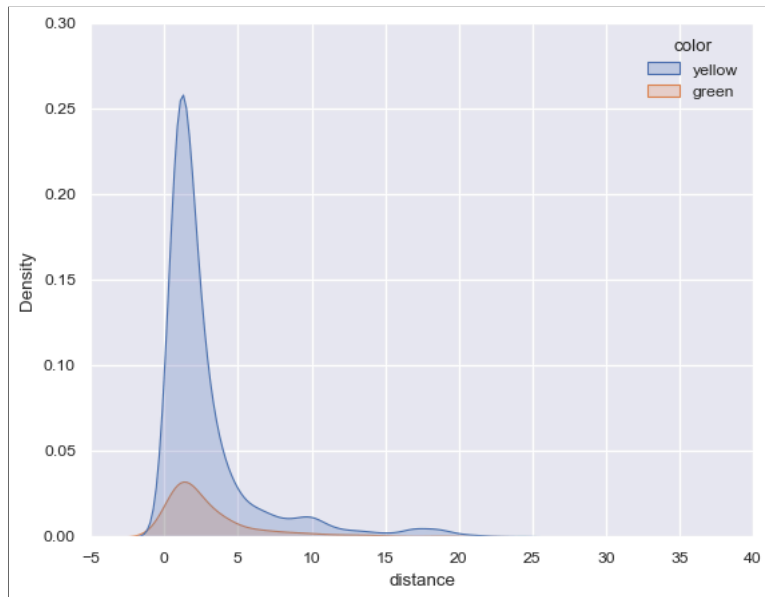
	pickup	dropoff	passengers	distance	fare	tip	tolls	total	color	payment	pickup_zone	dropoff_zone	pickup_id
0	2019-03-23 20:21:09	2019-03-23 20:27:24	1	1.60	7.0	2.15	0.0	12.95	yellow	credit card	Lenox Hill West	UN/Turtle Bay South	M.
1	2019-03-04 16:11:55	2019-03-04 16:19:00	1	0.79	5.0	0.00	0.0	9.30	yellow	cash	Upper West Side South	Upper West Side South	M.
2	2019-03-27 17:53:01	2019-03-27 18:00:25	1	1.37	7.5	2.36	0.0	14.16	yellow	credit card	Alphabet City	West Village	M.
3	2019-03-10 01:23:59	2019-03-10 01:49:51	1	7.70	27.0	6.15	0.0	36.95	yellow	credit card	Hudson Sq	Yorkville West	M.
4	2019-03-30 13:27:42	2019-03-30 13:37:14	3	2.16	9.0	1.10	0.0	13.40	yellow	credit card	Midtown East	Yorkville West	M.

There are two cab colors in this data, *green* and *yellow*. Which group of taxis travels more? Which color makes the most revenue? Let us look at some plots.

First, we will look at the distribution of `distance` for yellow and green taxis.

In [112]:

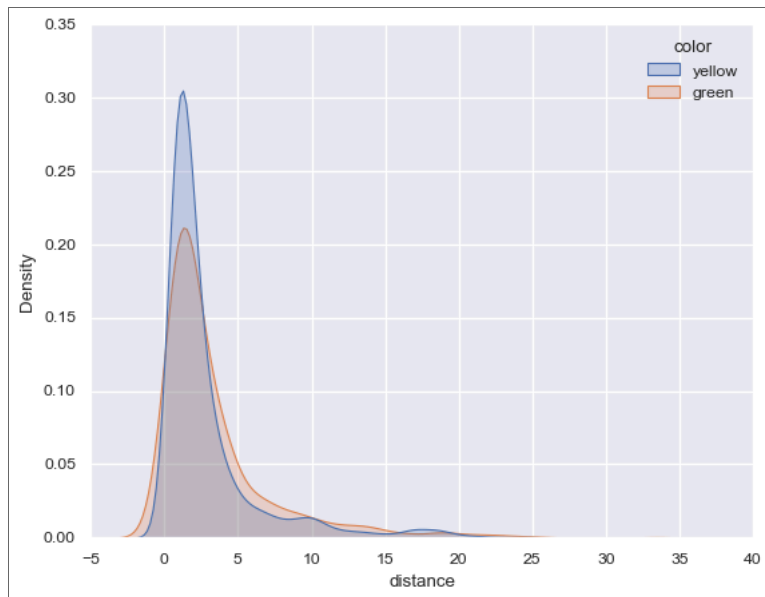
```
sns.kdeplot(data=taxis, x='distance', fill=True, hue='color');
```



There are way more yellow cabs than green ones. The distribution shows that difference. By default, the distributions are normalized jointly. If you are interested in investigating the proportion of yellow and green taxis independently, you can set the `common_norm` argument to `False`.

In [113]:

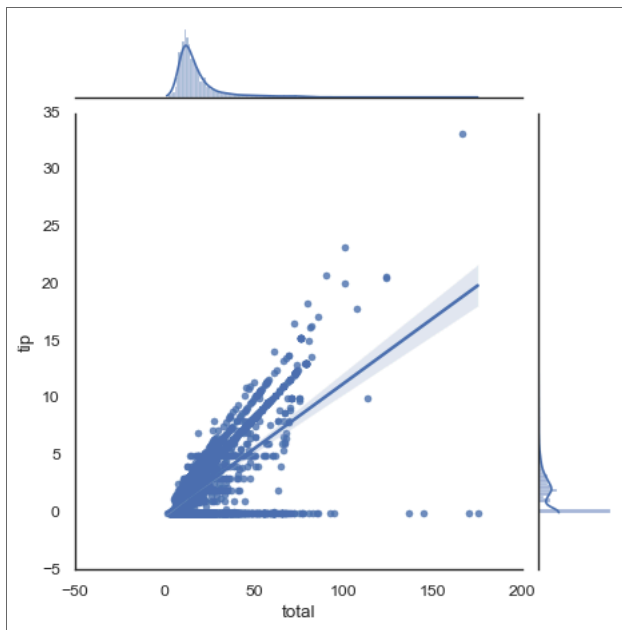
```
sns.kdeplot(data=taxis, x='distance', fill=True, hue='color', common_norm=False);
```



Let us check if the tip and total fare correlate by using a jointplot.

In [114]:

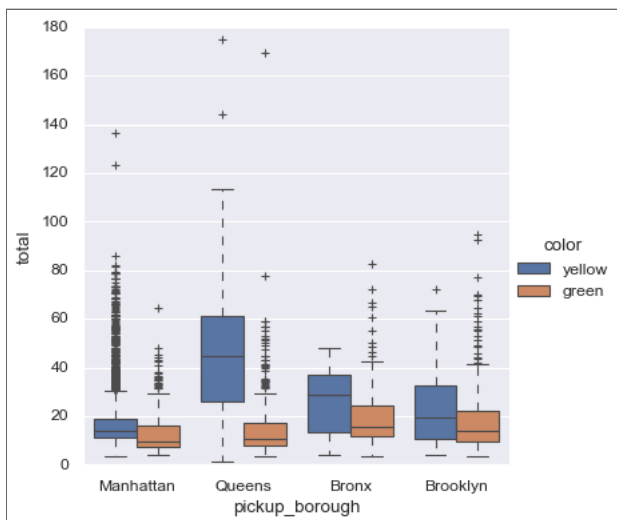
```
with sns.axes_style('white'):  
    sns.jointplot(x="total", y="tip", data=taxis, kind='reg')
```



Let us plot the total fare per pickup borough for each taxi color in a box plot.

In [115]:

```
sns.catplot(data=taxis, x='pickup_borough', hue='color', y='total', kind='box');
```



Now let us look at several variables paired up to identify possible correlations using PairGrid.

In [116]:

```
g = sns.PairGrid(taxis, vars=['fare', 'tip', 'distance', 'total'],  
                  hue='color', palette='RdBu_r')  
g.map(plt.scatter)  
g.add_legend();
```

